

✓ NDgpu — fused CUDA kernels: Phase 0/1/3 (Colab)

The claim under test. ndgpu is written against the NumPy API that CuPy mirrors, so CuPy dispatches *one CUDA kernel per array operation*. The 7-point diffusion stencil is six shifted multiply-adds — 13 kernels and 7 full-size temporaries per `apply` — and that `apply` is the innermost operation of every diffusion, SP3, SPN, DSA and CMFD solve. At the grid sizes this code actually runs (1e4–1e6 cells) each launch costs a few microseconds *regardless of grid size*, while the arithmetic takes microseconds. If that is right, the solvers are **launch-bound, not bandwidth-bound**, and collapsing the operation chains into single kernels is the win — not better memory layout.

This notebook measures whether that is true on *your* GPU, and what the fused kernels (`ndgpu.kernels`) actually buy.

phase	change	measured here
0	instrumentation (<code>ndgpu.profiling</code>)	cost of one launch; kernels per <code>apply</code>
1	fused 7-point stencil	<code>GroupOperator.apply</code> : 13 launches → 1
3	fused CG updates + reduction	3 dots + 3 vector updates → 3 + 2 kernels

Everything is A/B'd against the *identical* code path with fusion switched off (`ndgpu.kernels.set_fused(False)`), so each ratio isolates fusion and nothing else.

Go/no-go. Section 2 is a hard correctness gate — fused and unfused must agree to round-off, and end-to-end `k_eff` must agree to $|\Delta k| \leq 2e-8$. A preconditioner or kernel that moves `k` is a bug, not a trade-off. Do not read the timings if the gate fails.

Scope note: Phase 1 fuses the **Cartesian** `GroupOperator`. The triangular (`TriGroupOperator`, HP-MR) and hex stencils are untouched, so HP-MR cases are deliberately not in the tables below — the benchmarks here are Cartesian (bare box, IAEA-3D, C5G7-2D).

```
import os
try:
    from google.colab import files
    uploaded = files.upload()
    zip_name = next(iter(uploaded))
    get_ipython().run_line_magic("pip", f"install -q {zip_name}")
    try:
        import cupy
    except ImportError:
        get_ipython().run_line_magic("pip", "install -q cupy-cuda12x")
# Colab: upload dist/ndgpu-src.z
```

```
get_ipython().system("nvidia-smi -L")
except ImportError:
    pass
```

[Browse...](#) ndgpu-src.zip

ndgpu-src.zip(application/x-zip-compressed) - 285803 bytes, last modified: n/a - 100% done
 Saving ndgpu-src.zip to ndgpu-src.zip
 Installing build dependencies ... done
 Getting requirements to build wheel ... done
 Preparing metadata (pyproject.toml) ... done
 Building wheel for ndgpu (pyproject.toml) ... done
 GPU 0: Tesla T4 (UUID: GPU-49195f6f-c762-2613-c2bd-99f23031dd59)

```
import os
import time
import numpy as np

from ndgpu import Grid, Material, PWR_TWO_GROUP, get_backend, device_name, k
from ndgpu import DiffusionEigenSolver, SP3EigenSolver, SDP3EigenSolver
from ndgpu import kernels, profiling
from ndgpu.stencil import GroupOperator
from ndgpu.linalg import pcg, neumann_preconditioner

xp = get_backend("auto")
GPU = kernels.is_cupy(xp)
print("backend:", device_name(xp))
print("fused kernels active:", kernels.use_fused(xp))
if not GPU:
    print("\n*** No GPU: every A/B below will report a ratio of 1.0 (the fus
        "path is GPU-only). Switch the Colab runtime to a GPU. ***")

QUICK = bool(os.environ.get("NDGPU_QUICK"))

backend: cuda (cupy): Tesla T4
fused kernels active: True
```

1. Phase 0 — what does one kernel launch cost, and how many are there?

`profiling.launch_cost` times a trivial elementwise kernel on an 8-element array: no meaningful memory traffic, so what is left is the CUDA launch plus CuPy's per-operation Python dispatch. `profiling.effective_launches` then times an operation on a grid small enough to be *pure* overhead and divides — giving the number of kernels that operation launched, with no profiler and no privileges (`nsys` is not available in Colab).

Expect ≈ 13 for the unfused `GroupOperator.apply`, ≈ 1 fused. SP3 nests two of them plus the moment coupling; SDP3 nests four projections each containing a full stencil apply plus a dense 4×4 reaction — that one is the reason Phase 2 exists.

```
TINY = (6, 6, 6)          # small enough that only launch overhead is left

def tiny_ops():
    """One operator per method, built on a tiny grid, at their real complexi
    g = Grid(shape=TINY, size=(10.0, 10.0, 10.0))
    mat = PWR_TWO_GROUP
    out = {}
    out["GroupOperator (diffusion)"] = DiffusionEigenSolver(g, mat, device="
    out["SP3GroupOperator"] = SP3EigenSolver(g, mat, device="auto").ops[0]
    out["SDPNGroupOperator (order 3)"] = SDP3EigenSolver(g, mat, device="aut
    return out

def state_for(op, shape):
    """A right-shaped input: scalar for diffusion, (M, ...) for the block op
    M = getattr(op, "M", None) or (2 if hasattr(op, "moment2") else 1)
    s = shape if M == 1 else (M,) + tuple(shape)
    return xp.ones(s, dtype=np.float64)

tiny = profiling.launch_cost(xp)
print(f"cost of one trivial kernel: {tiny*1e6:.2f} us\n")

rows = []
for name, op in tiny_ops().items():
    u = state_for(op, TINY)
    r = {"operator": name}
    for tag, flag in (("unfused", False), ("fused", True)):
        prev = kernels.set_fused(flag)
        try:
            r[tag] = profiling.effective_launches(lambda: op.apply(u), xp,
                                                    tiny_cost=tiny, n_repeat=3)
        finally:
            kernels.set_fused(prev)
    rows.append(r)
profiling.report(rows, ["operator", "unfused", "fused"],
                  "kernels launched per apply (estimated)")
```

cost of one trivial kernel: 10.77 us

```
=== kernels launched per apply (estimated) ===
              operator  unfused  fused
GroupOperator (diffusion)    50.52   7.94
      SP3GroupOperator    187.2  25.67
SDPNGroupOperator (order 3)   738  195.2
```

✓ 2. Correctness gate

The fused kernel re-derives the flat index of each of the three face-coupling arrays, which are one cell short on *different* axes (`wx` is (nx-1, ny, nz), `wy` is (nx, ny-1, nz), `wz` is (nx, ny, nz-1)). That index arithmetic is already pinned against the slice form on CPU by `tests/verification/test_fused_kernels.py`, which transcribes the kernel body into NumPy. Here we check the *real* kernel, on the awkward cases: grids one cell thick on some axis, an excised-cell `active` mask, every boundary law, the cylindrical divergence form (`row_scale`), and a complex operator (the noise solver's).

The two paths are not bit-identical — the CUDA compiler contracts multiply-adds into FMAs — so the tolerance is round-off, not zero.

```
host = lambda a: np.asarray(a.get() if hasattr(a, "get") else a)

def max_rel(a, b):
    a, b = host(a), host(b)
    scale = max(np.abs(a).max(), np.abs(b).max(), 1e-300)
    return float(np.abs(a - b).max() / scale)

def check(label, build, shape, dtype=np.float64, seed=0):
    rng = np.random.default_rng(seed)
    op = build(rng)
    phi = xp.asarray(rng.standard_normal(shape).astype(dtype))
    prev = kernels.set_fused(False)
    try:
        ref = op.apply(phi)
        kernels.set_fused(True)
        got = op.apply(phi)
    finally:
        kernels.set_fused(prev)
    err = max_rel(ref, got)
    print(f"{label:<46} max rel err = {err:.3e}    {'OK' if err < 1e-12 else 'ERR'}")
    return err

def op_builder(shape, size=(10.0, 8.0, 6.0), geometry="cartesian", **kw):
    def build(rng):
        g = Grid(shape=shape, size=size, geometry=geometry)
        D = xp.asarray(0.5 + rng.random(shape))
        rem = xp.asarray(0.1 + rng.random(shape))
        return GroupOperator(xp, g, D, rem, **kw)
    return build

errs = []
for shp in [(9, 8, 7), (12, 1, 5), (1, 1, 16), (7, 7, 1)]:
    errs.append(check(f"cartesian {shp}" op_builder(shp) shp))
```

```

    errs.append(check("cartesian (shp)", op_builder(shp), shp))
for bc in ["zero-flux", "reflective", "vacuum", 0.4695]:
    shp = (9, 8, 7)
    errs.append(check(f"bc={bc!r}", op_builder(shp, bc=bc), shp, seed=1))

shp = (10, 9, 8)
mask = np.random.default_rng(2).random(shp) > 0.3
errs.append(check("active mask (excised cells)",
                  op_builder(shp, active=xp.asarray(mask), mask_bc="vacuum"))

shp = (16, 1, 12)
errs.append(check("cylindrical, volume-weighted (SPD)",
                  op_builder(shp, geometry="cylindrical"), shp, seed=3))
errs.append(check("cylindrical, divergence form (row_scale)",
                  op_builder(shp, geometry="cylindrical", symmetric=False),

def complex_builder(shape):
    def build(rng):
        g = Grid(shape=shape, size=(10.0, 8.0, 6.0))
        D = xp.asarray(0.5 + rng.random(shape))
        rem = xp.asarray((0.1 + rng.random(shape)) + 1j * rng.random(shape))
        return GroupOperator(xp, g, D, rem)
    return build

shp = (9, 8, 7)
errs.append(check("complex removal (noise solver)", complex_builder(shp), shp,
                  dtype=np.complex128, seed=5))

assert max(errs) < 1e-12, f"CORRECTNESS GATE FAILED: max rel err {max(errs):}"
print("\nstencil gate passed")

```

cartesian (9, 8, 7)	max rel err = 1.143e-16	OK
cartesian (12, 1, 5)	max rel err = 1.563e-16	OK
cartesian (1, 1, 16)	max rel err = 7.565e-17	OK
cartesian (7, 7, 1)	max rel err = 1.353e-16	OK
bc='zero-flux'	max rel err = 1.158e-16	OK
bc='reflective'	max rel err = 1.341e-16	OK
bc='vacuum'	max rel err = 1.341e-16	OK
bc=0.4695	max rel err = 1.341e-16	OK
active mask (excised cells)	max rel err = 8.925e-17	OK
cylindrical, volume-weighted (SPD)	max rel err = 1.267e-16	OK
cylindrical, divergence form (row_scale)	max rel err = 1.074e-16	OK
complex removal (noise solver)	max rel err = 0.000e+00	OK

stencil gate passed

```

# The Krylov kernels update their vectors *in place* through CuPy's out-para
# references, which is the part of ndgpu.kernels that CPU tests cannot exerc
# Check each against the expression it replaces, on device.
n = 4096
rng = np.random.default_rng(0)
xs, rs_, ps, aps, zs = (xp.asarray(rng.standard_normal(n)) for _ in range(5))

```

```

inv_diag = xp.asarray(1.0 / (2.0 + rng.random(n)))
alpha = xp.asarray(0.37) # 0-d device scalars, as pcg uses
beta = xp.asarray(-1.25)

kerr = {}
x_ref, r_ref = xs + alpha * ps, rs_ - alpha * aps
xf, rf = xs.copy(), rs_.copy()
kernels.cg_update(xp, xf, rf, ps, aps, alpha)
kerr["cg_update x"], kerr["cg_update r"] = max_rel(xf, x_ref), max_rel(rf, r_ref)

p_ref = zs + beta * ps
pf = ps.copy()
kernels.cg_direction(xp, pf, zs, beta)
kerr["cg_direction"] = max_rel(pf, p_ref)

zf = zs.copy()
kernels.neumann_step(xp, zf, rs_, aps, inv_diag)
kerr["neumann_step"] = max_rel(zf, zs + inv_diag * (rs_ - aps))

kerr["dot"] = max_rel(kernels.dot(xp, xs, rs_), xp.sum(xs * rs_))

for k, v in kerr.items():
    print(f"{k:<20} max rel err = {v:.3e}  {'OK' if v < 1e-12 else 'FAIL'}")
assert max(kerr.values()) < 1e-12, "KRYLOV KERNEL GATE FAILED"
print("\ngate passed – the timings below are for correct arithmetic")

cg_update x          max rel err = 1.158e-16   OK
cg_update r          max rel err = 1.187e-16   OK
cg_direction          max rel err = 1.362e-16   OK
neumann_step          max rel err = 1.915e-16   OK
dot                   max rel err = 1.371e-15   OK

gate passed – the timings below are for correct arithmetic

```

✓ 3. Phase 1 — the stencil apply vs. grid size

The signature of a launch-bound operation is a speedup that is **large on small grids and shrinks as the grid grows**: overhead is fixed, arithmetic is not. Once the fused kernel becomes bandwidth-bound, the remaining ~3–4× is the memory traffic saved by reading `phi` once with the seven stencil points served from cache, instead of seven separate streaming reads plus seven temporaries.

If the speedup is flat and small across the whole sweep, this GPU is *not* launch-bound at these sizes and the rest of the plan should be re-ordered around bandwidth instead.

```
SIZES = [(16.)*3, (32.)*3, (64.)*3] if QUICK else \
```

```

-----
[(8,)*3, (16,)*3, (32,)*3, (64,)*3, (96,)*3, (128,)*3]

rows = []
for shp in SIZES:
    rng = np.random.default_rng(0)
    g = Grid(shape=shp, size=(100.0, 100.0, 100.0))
    op = GroupOperator(xp, g, xp.asarray(0.5 + rng.random(shp)),
                      xp.asarray(0.1 + rng.random(shp)))
    phi = xp.asarray(rng.standard_normal(shp))
    n_rep = 50 if np.prod(shp) > 5e5 else 300
    ab = profiling.ab_compare(lambda: op.apply(phi), xp, n_repeat=n_rep)
    cells_n = int(np.prod(shp))
    rows.append(dict(shape="x".join(map(str, shp)), cells=cells_n,
                    unfused_us=ab["off"] * 1e6, fused_us=ab["on"] * 1e6,
                    speedup=ab["speedup"],
                    fused_Gcell_s=cells_n / ab["on"] / 1e9))
profiling.report(rows, ["shape", "cells", "unfused_us", "fused_us", "speedup",
                        "fused_Gcell_s"], "GroupOperator.apply")

```

```

=== GroupOperator.apply ===

```

shape	cells	unfused_us	fused_us	speedup	fused_Gcell_s
8x8x8	512	417.7	57.02	7.326	0.008979
16x16x16	4096	388.7	58.49	6.646	0.07003
32x32x32	32768	370.8	65.01	5.704	0.5041
64x64x64	262144	394.1	57.16	6.894	4.586
96x96x96	884736	1102	171.8	6.417	5.151
128x128x128	2097152	2559	399	6.413	5.256

✓ 4. Phase 3 — the CG inner loop

A CG iteration is three reductions and three vector updates on arrays that fit in cache: written as plain array expressions that is ~9 kernels and 3 full-size temporaries per iteration, almost entirely dispatch. `ndgpu.kernels` fuses the `x += ap; r -= αAp` pair into one kernel, `p ← z + βp` into one, and replaces `xp.sum(u*v)` with a `ReductionKernel` — deliberately *not* cuBLAS `dot`, since cuBLAS calls cannot be recorded into a CUDA graph and capturing this loop is the next step (Phase 4).

The coefficients stay 0-d **device** scalars; only the convergence test (`check_every`) touches the host. The iteration counts must be identical between the two legs — the arithmetic is the same, so a differing count would mean the fused kernels changed the recurrence.

```

def cg_case(shape, degree=0, check_every=1):
    rng = np.random.default_rng(0)
    g = Grid(shape=shape, size=(100.0, 100.0, 100.0))

```



```

op = GroupOperator(xp, g, xp.asarray(0.5 + rng.random(shape)),
                  xp.asarray(0.1 + rng.random(shape)))
b = xp.asarray(rng.standard_normal(shape))
x0 = xp.zeros_like(b)
pre = (neumann_preconditioner(op.apply, op.inv_diag, degree)
      if degree else None)
run = lambda: pcg(op.apply, b, x0, op.inv_diag, xp, rtol=1e-8,
                 maxiter=5000, precondition=pre, check_every=check_every)

return run

rows = []
for shape in [(32,)*3, (64,)*3] if QUICK else [(16,)*3, (32,)*3, (64,)*3, (
    for degree, check_every in [(0, 1), (0, 10), (6, 10)]:
        run = cg_case(shape, degree, check_every)
        prev = kernels.set_fused(False)
        try:
            _, it_off = run()
            t_off = profiling.timeit(run, xp, n_repeat=3, n_warmup=1)
            kernels.set_fused(True)
            _, it_on = run()
            t_on = profiling.timeit(run, xp, n_repeat=3, n_warmup=1)
        finally:
            kernels.set_fused(prev)
        assert it_off == it_on, (
            f"iteration count changed with fusion: {it_off} -> {it_on}")
        rows.append(dict(shape="x".join(map(str, shape)),
                        precondition=f"neumann-{degree}" if degree else "jacobi"
                        check_every=check_every, iters=it_on,
                        unfused_s=t_off, fused_s=t_on,
                        speedup=t_off / t_on if t_on else float("nan"))))
profiling.report(rows, ["shape", "precond", "check_every", "iters",
                       "unfused_s", "fused_s", "speedup"],
                 "PCG solve (rtol=1e-8), same iteration count both legs")

```

```

=== PCG solve (rtol=1e-8), same iteration count both legs ===

```

shape	precond	check_every	iters	unfused_s	fused_s	speedup
16x16x16	jacobi	1	10	0.01063	0.004968	2.14
16x16x16	jacobi	10	10	0.008098	0.003324	2.436
16x16x16	neumann-6	10	10	0.03919	0.01115	3.515
32x32x32	jacobi	1	16	0.01426	0.006305	2.261
32x32x32	jacobi	10	20	0.01559	0.01319	1.182
32x32x32	neumann-6	10	10	0.03948	0.008794	4.489
64x64x64	jacobi	1	27	0.02884	0.03007	0.9591
64x64x64	jacobi	10	30	0.0259	0.0173	1.497
64x64x64	neumann-6	10	10	0.03956	0.01196	3.308
96x96x96	jacobi	1	38	0.0778	0.0948	0.8207
96x96x96	jacobi	10	40	0.07646	0.07603	1.006
96x96x96	neumann-6	10	20	0.2033	0.07603	2.673

✓ 5. End to end — full k-eigenvalue solves

The number that matters. Same grid, same tolerances (`tol_k=1e-8`, `tol_source=1e-7`), best-of-3, one process; the only difference between the two legs is the `ndgpu.kernels` dispatch.

Accuracy is reported against a named reference in every row, and the two legs must agree to $|\Delta k| \leq 2e-8$ — that is the equivalence bound for a change that is supposed to be a pure re-association of the same arithmetic.

References: the bare box against the analytic `k_bare_box`; IAEA-3D against the classic 1.02903; C5G7-2D against the transport reference (the diffusion discrepancy there is physics, not solver error — what matters is that it does not *move*).

```
from ndgpu.benchmarks import build_iaea, build_c5g7_2d, K_REFERENCE_2D

TOL = dict(tol_k=1e-8, tol_source=1e-7)
BEST_OF = 1 if QUICK else 3

def best_of(make_solver, n=BEST_OF):
    """Fastest of n identical solves; returns (result, seconds)."""
    best, res = float("inf"), None
    for _ in range(n):
        r = make_solver().solve(**TOL)
        assert r.converged, r
        best, res = min(best, r.solve_seconds), r
    return res, best

def ab_solve(label, make_solver, k_ref, ref_name):
    out = {}
    for tag, flag in (("unfused", False), ("fused", True)):
        prev = kernels.set_fused(flag)
        try:
            make_solver().solve(max_outer=2, tol_k=0) # warm-up, not time
            r, t = best_of(make_solver)
        finally:
            kernels.set_fused(prev)
        out[tag] = (r, t)
    r_off, t_off = out["unfused"]
    r_on, t_on = out["fused"]
    dk = abs(r_on.k_eff - r_off.k_eff)
    assert dk <= 2e-8, f"{label}: fusion moved k_eff by {dk:.2e} (bound 2e-8)"
    return dict(case=label, k_eff=f"{r_on.k_eff:.7f}",
                dpcm=f"{1e5 * (r_on.k_eff - k_ref):+.1f}", ref=ref_name,
                outers=r_on.outer_iterations, inners=r_on.inner_iterations,
                unfused_s=t_off, fused_s=t_on, speedup=t_off / t_on)

rows = []
```

```

box = (150.0, 150.0, 150.0)
for n in ([48] if QUICK else [48, 96, 144]):
    g = Grid(shape=(n, n, n), size=box)
    rows.append(ab_solve(f"bare box {n}^3, diffusion",
                        lambda g=g: DiffusionEigenSolver(g, PWR_TWO_GROUP, d
                        k_bare_box(PWR_TWO_GROUP, box), "analytic"))

p = build_iaea(cells_per_node=2 if QUICK else 3)
rows.append(ab_solve(f"IAEA-3D {p.grid.shape}, diffusion",
                    lambda p=p: DiffusionEigenSolver(
                        p.grid, p.materials, p.material_map, bc=p.bc,
                        mask_bc=p.mask_bc, active=p.active, device="auto"),
                    1.02903, "IAEA-3D ref"))

c = build_c5g7_2d(cells_per_pin=2)
for name, cls in [("diffusion", DiffusionEigenSolver), ("SP3", SP3EigenSolver)]:
    rows.append(ab_solve(f"C5G7-2D {c.grid.shape[:2]}, {name}",
                        lambda cls=cls: cls(c.grid, c.materials, c.material_map,
                        bc=c.bc, device="auto"),
                        K_REFERENCE_2D, "C5G7 transport"))

profiling.report(rows, ["case", "k_eff", "dpcm", "ref", "outers", "inners",
                        "unfused_s", "fused_s", "speedup"],
                f"end-to-end, tol_k={TOL['tol_k']}, tol_source={TOL['tol_source']},
                f"best of {BEST_OF}")

```

```
=== end-to-end, tol_k=1e-08, tol_source=1e-07, best of 3 ===
```

	case	k_eff	dpcm	ref	outers	i
	bare box 48^3, diffusion	1.2122702	+1.9	analytic	22	
	bare box 96^3, diffusion	1.2122556	+0.5	analytic	22	
	bare box 144^3, diffusion	1.2122529	+0.2	analytic	21	
	IAEA-3D (51, 51, 57), diffusion	1.0286555	-37.5	IAEA-3D ref	60	
	C5G7-2D (102, 102), diffusion	1.1865010	-4.9	C5G7 transport	54	
	C5G7-2D (102, 102), SP3	1.1877869	+123.7	C5G7 transport	53	

6. Reading the result, and what comes next

- **Section 1** tells you whether this GPU is launch-bound at these sizes. If the unfused `apply` really costs ~13 launches and the fused one ~1, the model behind the whole plan holds.
- **Section 3** should show the speedup *decaying* with grid size toward the bandwidth-bound ratio. Where it flattens out is the crossover between the two regimes, and it sets how much Phase 4 (CUDA graph capture, which only attacks launch overhead) is worth.
- **Section 5** is the honest end-to-end number. It will be smaller than the micro-

benchmark: a solve also spends time in the group-source assembly (`solver.py`, G^2 kernel launches per outer — Phase 6) and in reductions. A large gap between section 3 and section 5 is itself the argument for Phase 6.

Still unimplemented from `docs/gpu_kernel_optimization_plan.md`:

- **Phase 2** — fuse the SDPN block apply (~145 launches $\rightarrow$ 1). The biggest single item, and the path the HP-MR drum work runs on. Section 1 sizes it.
- **Phase 4** — capture the CG inner loop as a CUDA graph. Phase 3 already made the loop allocation-free between convergence checks, which is the prerequisite.
- **Phase 5** — remove the host round-trips in the Cartesian S_N path (`sn.py`), which currently does a $D2H \rightarrow \text{scipy LU} \rightarrow H2D$ per source iteration.
- **Phase 6** — batch the per-group loops.
- **Precision** — `sn.py`/`tri_sn.py` are hard-wired float64. On a consumer GeForce card FP64 runs at 1/32–1/64 of FP32, which would outweigh every fusion item here. Worth checking what card this notebook just ran on.